In [45]:
```
# importing all the dependencies
import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score
```

In [46]:
```
# data collection and processing
# loading the csv data to a pandas data frame
heart_data = pd.read_csv('Downloads/heart_disease_data.csv')
# printing the first 5 rows of the data set
heart_data.head()
```

Out[46]:

	age	sex	cp	trestbps	chol	fbs	restecg	thalach	exang	oldpeak	slope	ca	thal
0	63	1	3	145	233	1	0	150	0	2.3	0	0	1
1	37	1	2	130	250	0	1	187	0	3.5	0	0	2
2	41	0	1	130	204	0	0	172	0	1.4	2	0	2
3	56	1	1	120	236	0	1	178	0	0.8	2	0	2
4	57	0	0	120	354	0	1	163	1	0.6	2	0	2

In [47]:
```
# printing the last 5 rows of the data set
heart_data.tail()
```

Out[47]:

	age	sex	cp	trestbps	chol	fbs	restecg	thalach	exang	oldpeak	slope	ca	th
298	57	0	0	140	241	0	1	123	1	0.2	1	0	
299	45	1	3	110	264	0	1	132	0	1.2	1	0	
300	68	1	0	144	193	1	1	141	0	3.4	1	2	
301	57	1	0	130	131	0	1	115	1	1.2	1	1	
302	57	0	1	130	236	0	0	174	0	0.0	1	1	

In [48]:
```
# now we gonna check number of rows and column in  the data set.
heart_data.shape
```

Out[48]:
```
(303, 14)
```

In [49]:
```
# getting more info about the data
heart_data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 303 entries, 0 to 302
Data columns (total 14 columns):
 #   Column    Non-Null Count  Dtype  
---  ------    --------------  -----  
 0   age       303 non-null    int64  
 1   sex       303 non-null    int64  
 2   cp        303 non-null    int64  
 3   trestbps  303 non-null    int64  
 4   chol      303 non-null    int64  
 5   fbs       303 non-null    int64  
 6   restecg   303 non-null    int64  
 7   thalach   303 non-null    int64  
 8   exang     303 non-null    int64  
 9   oldpeak   303 non-null    float64
 10  slope     303 non-null    int64  
 11  ca        303 non-null    int64  
 12  thal      303 non-null    int64  
 13  target    303 non-null    int64  
dtypes: float64(1), int64(13)
memory usage: 33.3 KB
```

In [50]:
```
# checcking for missing value.
heart_data.isnull().sum()
```

Out[50]:
```
age         0
sex         0
cp          0
trestbps    0
chol        0
fbs         0
restecg     0
thalach     0
exang       0
oldpeak     0
slope       0
ca          0
thal        0
target      0
dtype: int64
```

In [51]:
```
# now we gonna get some statistical measures of  our data
heart_data.describe()
```

Out[51]:

	age	sex	cp	trestbps	chol	fbs	reste
count	303.000000	303.000000	303.000000	303.000000	303.000000	303.000000	303.0000
mean	54.366337	0.683168	0.966997	131.623762	246.264026	0.148515	0.5280
std	9.082101	0.466011	1.032052	17.538143	51.830751	0.356198	0.5258
min	29.000000	0.000000	0.000000	94.000000	126.000000	0.000000	0.0000
25%	47.500000	0.000000	0.000000	120.000000	211.000000	0.000000	0.0000
50%	55.000000	1.000000	1.000000	130.000000	240.000000	0.000000	1.0000
75%	61.000000	1.000000	2.000000	140.000000	274.500000	0.000000	1.0000
max	77.000000	1.000000	3.000000	200.000000	564.000000	1.000000	2.0000

In [52]:
```
# checking the distribution of target variable.
heart_data['target'].value_counts()
# in the below output 1 represent defective heart and 0 represents healthy heart
```

Out[52]:
```
target
1    165
0    138
Name: count, dtype: int64
```

In [53]:
```
# splitting the features and target
x = heart_data.drop( columns ='target', axis=1)
y = heart_data['target']
print(x)
print(y)
```

```
     age  sex  cp  trestbps  chol  fbs  restecg  thalach  exang  oldpeak  \
0     63    1   3       145   233    1        0      150      0      2.3
1     37    1   2       130   250    0        1      187      0      3.5
2     41    0   1       130   204    0        0      172      0      1.4
3     56    1   1       120   236    0        1      178      0      0.8
4     57    0   0       120   354    0        1      163      1      0.6
..   ...  ...  ..       ...   ...  ...      ...      ...    ...      ...
298   57    0   0       140   241    0        1      123      1      0.2
299   45    1   3       110   264    0        1      132      0      1.2
300   68    1   0       144   193    1        1      141      0      3.4
301   57    1   0       130   131    0        1      115      1      1.2
302   57    0   1       130   236    0        0      174      0      0.0

     slope  ca  thal
0        0   0     1
1        0   0     2
2        2   0     2
3        2   0     2
4        2   0     2
..     ...  ..   ...
298      1   0     3
299      1   0     3
300      1   2     3
301      1   1     3
302      1   1     2

[303 rows x 13 columns]
0      1
1      1
2      1
3      1
4      1
      ..
298    0
299    0
300    0
301    0
302    0
Name: target, Length: 303, dtype: int64
```

In [54]:
```
# splitting data into train and test data
x_train, x_test, y_train, y_test = train_test_split(x,y, test_size = 0.2, strati
print(x.shape, x_train.shape, x_test.shape)
```

```
(303, 13) (242, 13) (61, 13)
```

In [66]:
```
# model training
# logistic regression.
model = LogisticRegression(max_iter=1000)
# training the logistic regression model with trainoing data.
model.fit(x_train.values,y_train)
```

Out[66]:

▾ LogisticRegression

▶ Parameters

In [56]:
```
# model evaluation
# accuracy score on training data.
x_train_prediction = model.predict(x_train)
```

```
training_data_accuracy = accuracy_score(x_train_prediction , y_train)
print('Accuracy on Training Data: ', training_data_accuracy)
```

```
Accuracy on Training Data:  0.8512396694214877
```

In [57]:
```
# accuracy score on test data.
x_test_prediction = model.predict(x_test)
test_data_accuracy = accuracy_score(x_test_prediction , y_test)
print('Accuracy on Test Data: ', test_data_accuracy)
```

```
Accuracy on Test Data:  0.819672131147541
```

In [68]:
```
# BUILDING A PREDICTIVE SYSTEM.
input_data= (60,1,0,130,206,0,0,132,1,2.4,1,2,3)
# we need to change the input data to a numpy array.
input_data_as_numpy_array = np.asarray(input_data)
# reshape the numpy array as we are predicting for only on instance
input_data_reshaped = input_data_as_numpy_array.reshape(1,-1)

prediction = model.predict(input_data_reshaped)
print(prediction)

if (prediction[0] == 0):
    print('The Person does not have a Heart Disease')
else:
    print('The Person has Heart Disease')
```

```
[0]
The Person does not have a Heart Disease
```